

1. MapReduce K-Means Algorithm

The objective of this lab was to implement the K-Means clustering algorithm using the Hadoop MapReduce framework. The algorithm groups similar data points into clusters based on the Euclidean distance between each point and the cluster centroids.

The implementation was tested using the Iris dataset. The initial centroids provided in the assignment were:

- Cluster 0: (5, 3, 1, 0)
- Cluster 1: (5, 2, 4, 1)
- Cluster 2: (6, 3, 5, 2)

The algorithm was implemented using four Java classes:

Centroid.java

This class stores the centroid values in a `double[]` array and calculates the Euclidean distance between a data point and the centroid. Using an array instead of fixed variables makes the implementation generalized because it can work with any number of features.

KMeansMapper.java

The Mapper reads one point from the input dataset at a time.

For each point, it:

- Converts the feature values into a double array.
- Calculates the Euclidean distance between the point and each centroid.
- Finds the nearest centroid.
- Outputs the cluster ID as the key and the data point as the value.

Example output from the Mapper:

```
0 5.1,3.5,1.4,0.2
```

KMeansReducer.java

The Reducer receives all points assigned to the same cluster.

For each cluster, it:

- Adds all feature values.
- Counts the number of points.

- Computes the average of every feature.
- Outputs the new centroid.

This average becomes the updated centroid after one iteration of the K-Means algorithm.

KMeansDriver.java

The Driver configures the Hadoop job by specifying:

- Mapper class
- Reducer class
- Input path
- Output path

The Driver then submits the MapReduce job and waits until it finishes successfully.

2. Challenges Faced and Solutions

Several challenges were encountered during the implementation of this lab.

Challenge 1: JAVA_HOME was not configured

When starting Hadoop, the following error appeared:

JAVA_HOME is not set and could not be found.

Solution

The JAVA_HOME variable was configured correctly inside hadoop-env.sh, and Hadoop services were restarted successfully.

Challenge 2: Hadoop services were not running

Initially, the NameNode and DataNode were not running correctly, which caused connection errors while accessing HDFS.

Solution

The NameNode was formatted, Hadoop services were restarted using:

start-dfs.sh

start-yarn.sh

After restarting, the jps command confirmed that all required Hadoop processes were running successfully.

Challenge 3: NodeManager was not registering

The MapReduce job remained in the ACCEPTED state because the NodeManager was not available.

Solution

The NodeManager configuration was corrected, and additional YARN settings were added to yarn-site.xml. After restarting YARN, the NodeManager registered successfully with the ResourceManager.

Challenge 4: MRAppMaster ClassNotFoundException

During the first execution of the MapReduce job, Hadoop produced the following error:

Could not find or load main class

org.apache.hadoop.mapreduce.v2.app.MRAppMaster

Solution

The missing HADOOP_MAPRED_HOME properties were added to mapred-site.xml as required. After restarting Hadoop, the MapReduce application executed correctly.

3. Evaluation Results (Single Node Cluster)

The implementation was tested on a single-node Hadoop cluster running on Ubuntu inside Oracle VirtualBox.

The application compiled successfully and the JAR file was created without errors.

The MapReduce job completed successfully, and Hadoop produced the following output centroids:

Cluster 0

5.0060000000000001,3.4180000000000006,1.4640000000000004,0.244

Cluster 1

5.466666666666667,2.538095238095238,3.847619047619047,1.1809523809523812

Cluster 2

6.473417721518987,2.9607594936708863,5.187341772151897,1.8075949367088608

The successful execution of the program confirmed that:

- Hadoop MapReduce was configured correctly.
- The Mapper correctly assigned each data point to the nearest centroid.
- The Reducer successfully calculated the new centroid for each cluster.
- The implementation supports feature vectors of different sizes because it stores the features in arrays instead of fixed variables.

The application completed successfully on a one-node Hadoop cluster, satisfying the requirements of the lab.

Conclusion

In this lab, a parallel implementation of the K-Means clustering algorithm was developed using the Hadoop MapReduce framework. The Mapper assigned each data point to the closest centroid using the Euclidean distance, while the Reducer calculated the new centroid by averaging all points assigned to each cluster. The implementation was generalized to support different feature vector sizes by using arrays instead of fixed variables.

The program was successfully executed on a single-node Hadoop cluster, and the generated output demonstrated that the MapReduce implementation correctly performed one iteration of the K-Means clustering algorithm.

Screenshots:

1-Hadoop Processes (jps)

```
salma-rabie@salma-rabie-VirtualBox:~$ jps
4929 Jps
4616 ResourceManager
3913 NameNode
4122 DataNode
4748 NodeManager
4366 SecondaryNameNode
```

2- Project Folder

```
salma-rabie@salma-rabie-VirtualBox:~/KMeans$ ls
Centroid.java  iris.data      kmeans.jar      KMeansReducer.java
classes        KMeansDriver.java  KMeansMapper.java  output
```

3-Compilation

```
salma-rabie@salma-rabie-VirtualBox:~/KMeans$ rm -rf classes
mkdir classes
salma-rabie@salma-rabie-VirtualBox:~/KMeans$ javac -classpath `hadoop classpath` \
\
-d classes \
Centroid.java \
KMeansMapper.java \
KMeansReducer.java \
KMeansDriver.java
salma-rabie@salma-rabie-VirtualBox:~/KMeans$ ls classes
Centroid.class  KMeansDriver.class  KMeansMapper.class  KMeansReducer.class
```

4-Creating the JAR

```
salma-rabie@salma-rabie-VirtualBox:~/KMeans$ jar -cvf kmeans.jar -C classes/ .
added manifest
adding: KMeansMapper.class(in = 2072) (out= 972)(deflated 53%)
adding: KMeansReducer.class(in = 2103) (out= 958)(deflated 54%)
adding: Centroid.class(in = 724) (out= 487)(deflated 32%)
adding: KMeansDriver.class(in = 1314) (out= 726)(deflated 44%)
```

5-Running the Job

```
salma-rabie@salma-rabie-VirtualBox:~/KMeans$ hadoop jar kmeans.jar KMeansDriver /user/salma-rabie/kmeans /user/salma-rabie/output
2026-07-28 01:34:17,569 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using built-in-java classes where applicable
2026-07-28 01:34:18,882 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-07-28 01:34:19,638 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-07-28 01:34:19,723 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/salma-rabie/.staging/job_1785190457907_0002
2026-07-28 01:34:20,660 INFO input.FileInputFormat: Total input files to process : 1
2026-07-28 01:34:20,909 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-28 01:34:21,351 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1785190457907_0002
2026-07-28 01:34:21,352 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-28 01:34:21,786 INFO conf.Configuration: resource-types.xml not found
2026-07-28 01:34:21,786 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-28 01:34:21,975 INFO impl.YarnClientImpl: Submitted application application_1785190457907_0002
2026-07-28 01:34:22,051 INFO mapreduce.Job: The url to track the job: http://salma-rabie-VirtualBox:8088/proxy/application_1785190457907_0002/
2026-07-28 01:34:22,053 INFO mapreduce.Job: Running job: job_1785190457907_0002
2026-07-28 01:34:32,598 INFO mapreduce.Job: Job job_1785190457907_0002 running in uber mode : false
2026-07-28 01:34:32,600 INFO mapreduce.Job: map 0% reduce 0%
2026-07-28 01:34:39,906 INFO mapreduce.Job: map 100% reduce 0%
2026-07-28 01:34:52,505 INFO mapreduce.Job: map 100% reduce 100%
2026-07-28 01:34:55,794 INFO mapreduce.Job: Job job_1785190457907_0002 completed successfully
2026-07-28 01:34:56,182 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=3006
    FILE: Number of bytes written=624411
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=2511
    HDFS: Number of bytes written=215
    HDFS: Number of read operations=8
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=1
    Launched reduce tasks=1
    Data-local map tasks=1
    Total time spent by all maps in occupied slots (ms)=4883
    Total time spent by all reduces in occupied slots (ms)=9770
    Total time spent by all map tasks (ms)=4883
    Total time spent by all reduce tasks (ms)=9770
    Total vcore-milliseconds taken by all map tasks=4883
    Total vcore-milliseconds taken by all reduce tasks=9770
    Total megabyte-milliseconds taken by all map tasks=5000192
    Total megabyte-milliseconds taken by all reduce tasks=10004480
  Map-Reduce Framework
    Map input records=151
    Map output records=150
    Map output bytes=2700
    Map output materialized bytes=3006
    Input split bytes=110
    Combine input records=0
    Combine output records=0
    Reduce input groups=3
    Reduce shuffle bytes=3006
    Reduce input records=150
    Reduce output records=3
    Spilled Records=300
    Shuffled Maps =1
    Failed Shuffles=0
```

```

Failed Shuffles=0
Merged Map outputs=1
GC time elapsed (ms)=164
CPU time spent (ms)=3830
Physical memory (bytes) snapshot=565088256
Virtual memory (bytes) snapshot=5491027968
Total committed heap usage (bytes)=363855872
Peak Map Physical memory (bytes)=331067392
Peak Map Virtual memory (bytes)=2744307712
Peak Reduce Physical memory (bytes)=234020864
Peak Reduce Virtual memory (bytes)=2746720256

Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0

File Input Format Counters
  Bytes Read=2401
File Output Format Counters
  Bytes Written=215

```

7-Output File

```

salma-rabie@salma-rabie-VirtualBox:~/KMeans$ hdfs dfs -cat /user/salma-rabie/output/part-r-00000
2026-07-28 01:35:34,077 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using built
in-java classes where applicable
0      5.0060000000000001,3.4180000000000006,1.4640000000000004,0.244
1      5.466666666666667,2.538095238095238,3.847619047619047,1.1809523809523812
2      6.473417721518987,2.9607594936708863,5.187341772151897,1.8075949367088608

```

8- HDFS Output Folder

```

salma-rabie@salma-rabie-VirtualBox:~/KMeans$ hdfs dfs -ls /user/salma-rabie/output
2026-07-28 01:46:27,014 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using built
in-java classes where applicable
Found 2 items
-rw-r--r--  1 salma-rabie supergroup          0 2026-07-28 01:34 /user/salma-rabie/output/_SUCCESS
-rw-r--r--  1 salma-rabie supergroup       215 2026-07-28 01:34 /user/salma-rabie/output/part-r-00000

```